



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU
STRUKTURE I REŽIMI RADA ELEKTROENERGETSKIH
SISTEMA
AKADEMSKA 2025/26. GODINA

Konvertor nelinearnih punih tokova snaga

DOKUMENTACIJA

Student:

Amer Šabić

Sarajevo, juli 2026.

Sadržaj

Popis slika	ii
Popis tabela	ii
1 Uvod	1
2 Osnove <i>natID</i> GUI frameworka	2
2.1 Hijerarhija klasa: Window, View, Control	2
2.2 Rasporedi (layout sistem)	2
2.3 Događaji i <code>onClick</code> povratni pozivi	3
2.4 Prozori za dijaloge datoteka	3
3 Arhitektura aplikacije	5
3.1 Ulazna tačka: klasa <code>Plugin</code>	5
3.1.1 Tok podataka kroz aplikaciju	6
4 Korišćenje aplikacije	7
4.1 Glavni prozor (Converter tab)	7
4.2 Prozor za kreiranje modela	9
4.2.1 Odabir veličina za iscertavanje	10
4.2.2 Parametri generatora (GEN fajl)	10
4.2.3 Poremećaji iz fajla naspram ručnog unosa	11
4.3 Format ulaznih fajlova	12
4.3.1 GEN fajl (dinamički parametri generatora)	12
4.3.2 PLOT/BUS fajl (poremećaji tereta)	12
5 Algoritam generisanja modela	14
5.1 Parsiranje Matpower fajla	14
5.2 Građenje Y-bus matrice	15
5.3 Generisanje <code>.dmodl</code> sadržaja	16

Popis slika

3.1	Tok podataka od odabira fajla do generisanog modela.	6
4.1	Izgled glavnog prozora	7
4.2	Prozor za podešavanje modela	9

1 Uvod

Vjerovatno i najznačajniji dio rada je napraviti aplikaciju koja će omogućiti korisniku da simulira ponašanje nekog elektroenergetskog sistema zadajući konkretne pobude u određenim vremenskim trenucima. Za razliku od izvještaja seminarskog rada, koji se bavio matematičkim modelom generatora i teorijskom pozadinom tokova snaga, ovo je **tehnička dokumentacija** – fokusira se na sam kod, arhitekturu aplikacije i algoritme kojima se Matpower ulazni fajl pretvara u dinamički `dTwin` model. Aplikacija je napravljena u *natID* razvojnom okruženju, korištenjem njegovog `mu` GUI frameworka u jeziku C++.

2 Osnove *natID* GUI frameworka

Prije opisa same aplikacije, potrebno je razumjeti nekoliko osnovnih gradivnih blokova frameworka koji se koriste kroz cijeli kod.

2.1 Hijerarhija klasa: Window, View, Control

Framework razlikuje tri osnovna nivoa GUI elemenata:

- **gui::Window** – pravi operativni sistemski prozor (ima naslovnu traku, može se pomjerati, zatvarati, minimizirati). Svaki prozor u aplikaciji nasljeđuje **gui::Window** i mora postaviti tzv. *central view* metodom **setCentralView(...)**.
- **gui::View** – pravougaoni sadržajni kontejner *unutar* prozora (ili unutar drugog View-a, npr. unutar taba). View sadrži jedan *layout* (raspored) i skup kontrola. Svaki ekran u aplikaciji (Converter tab, Info tab, prozor za kreiranje modela) implementiran je kao zasebna klasa izvedena iz **gui::View**.
- **gui::Control** (i njegovi izvedeni tipovi: **Label**, **Button**, **LineEdit**, **NumericEdit**, **CheckBox**, **TextEdit**, **ProgressIndicator**) – pojedinačni interaktivni ili prikazni elementi.

Svaka klasa u aplikaciji (npr. **ViewConv**, **ViewCreateModel**) prati isti obrazac: kontrole se deklariraju kao **članovi klase** (ne alociraju se dinamički pokazivačima, osim kada je broj kontrola poznat tek u trenutku izvršavanja – npr. checkbox za svaki čvor sistema, gdje se koristi dinamička lista pokazivača), inicijalizuju u konstruktoru, a zatim se slažu u *layout* objekat koji se konstruktoru dodjeljuje pozivom **setLayout(&_layout)**.

2.2 Rasporedi (layout sistem)

Umjesto ručnog pozicioniranja svakog elementa preko piksela, framework nudi tri osnovna tipa rasporeda koja se koriste kroz cijelu aplikaciju:

- **gui::VerticalLayout** – slaže elemente jedan ispod drugog, redom kojim su dodani operatorom «. Metod **appendSpace(n)** dodaje fiksni razmak (prazan red) – koristi se za vizuelno odvajanje sekcija.
- **gui::HorizontalLayout** – slaže elemente jedan pored drugog u istom redu; koristi se npr. za grupisanje dugmadi (Info/Convert) ili checkbox-ova (grafici) u jedan red umjesto da svaki zauzima poseban red.

- **gui::GridLayout** (uz pomoćnu klasu **gui::GridComposer**) – pravi tabelu sa stvarnim, poravnatim kolonama. Ovo je bilo neophodno za tabelu poremećaja (§??): dvije nezavisne **HorizontalLayout** vrste ne garantuju da će kolone iz različitih redova biti poravnate, dok **GridLayout** to garantuje jer sve ćelije dijele isti proračun širine kolone.

Primjer tipičnog obrasca korištenog kroz cijelu aplikaciju:

```
1 _lblPlotTypes.setFont(gui::Font::ID::SystemLargerBold);
2 _hlPlotTypes << _cbSpeeds << _cbPowers << _cbAngles <<
   _cbVoltages;
3 _v1 << _lblPlotTypes << _hlPlotTypes;
4 setLayout(&_v1);
```

2.3 Događaji i onClick povratni pozivi

Framework koristi *event-driven* model: umjesto nasljeđivanja i override-ovanja metoda za obradu klika, svaka kontrola nudi metod **onClick(...)** koji prima C++ lambda izraz. Lambda hvata **this** pokazivač (`[this] {...}`), čime dobija pristup svim ostalim članovima klase (drugim kontrolama, stanju):

```
1 _btnConvert.onClick([this] {
2     td::String inputFileName = _editFnIn.getText();
3     if (inputFileName.isEmpty()) {
4         _editStatus = "ERROR! Empty input file name";
5         _editStatus.setTextColor(td::Accent::Error);
6         return;
7     }
8     // ... dalje logika ...
9 });
```

Ovaj obrazac se ponavlja za svako dugme, checkbox i polje za unos u cijeloj aplikaciji.

2.4 Prozori za dijaloge datoteka

Za odabir/snimanje fajlova framework nudi statičke metode **gui::OpenFileDialog::show(...)** i **gui::SaveFileDialog::show(...)**, koje otvaraju sistemski dijalog i, nakon što korisnik odabere fajl (ili odustane), pozivaju predati *callback* sa rezultatom:

```
1 gui::OpenFileDialog::show(this, tr("openEQModel"), "*.m", _wndID
   + 1000,
2     [this](gui::FileDialog* pDlg) {
3         if (pDlg->getStatus() == gui::FileDialog::Status::OK)
4             {
5                 td::String fn = pDlg->getFileName();
6                 if (!fn.isEmpty()) { _editFnIn = fn; _editFnIn.
setFocus(); }
7             }
8     });
```

Svaki dijalog zahtijeva jedinstveni ID prozora (`_wndID + offset`), kako bi framework interno razlikovao više istovremeno otvorenih dijaloga – u aplikaciji su korišteni razmaknuti brojevi (+1000, +2000, +3000...) da se izbjegnu kolizije između različitih dijaloga i podprozora otvorenih iz iste klase.

3 Arhitektura aplikacije

3.1 Ulazna tačka: klasa Plugin

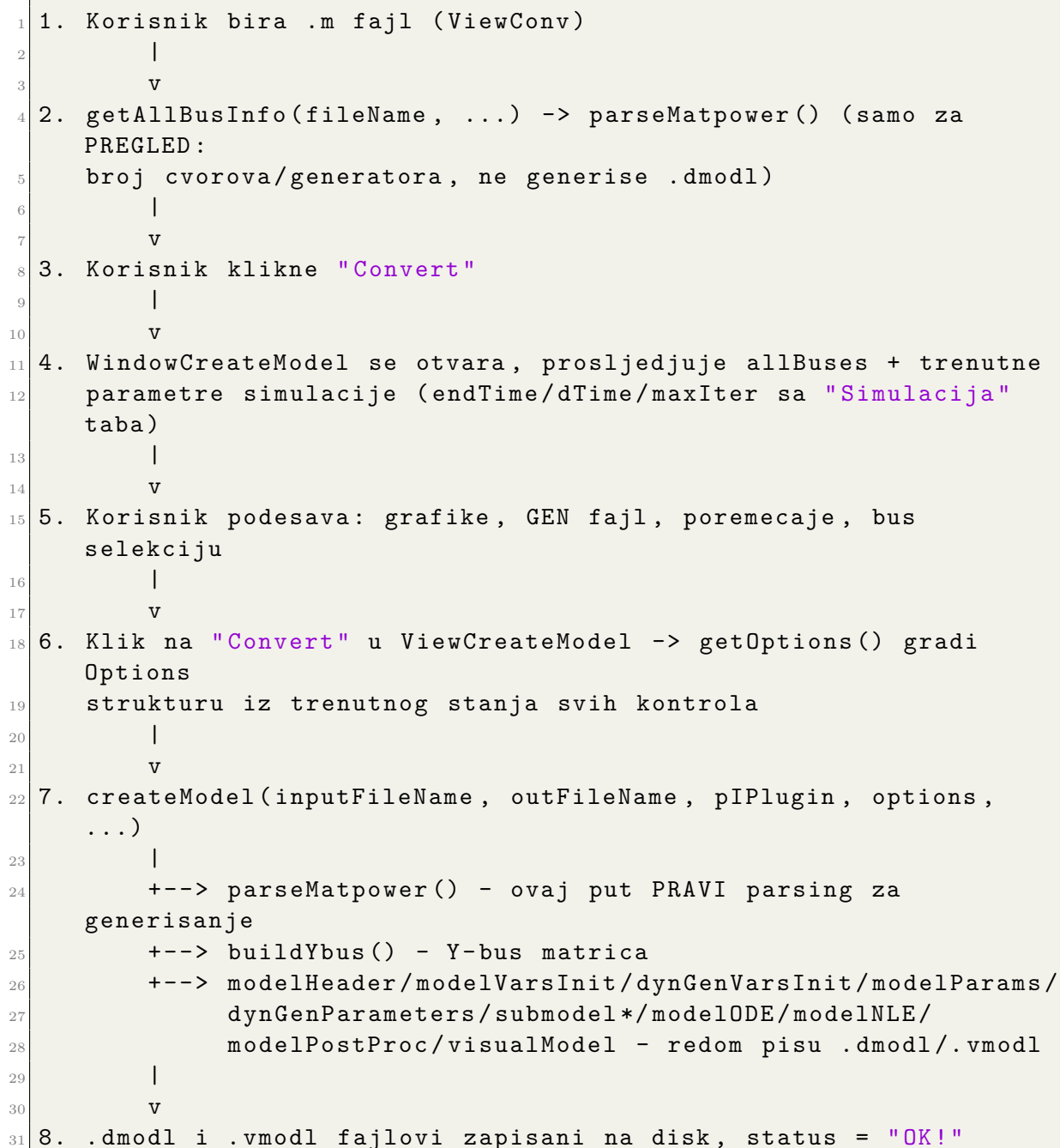
Cijela aplikacija je organizovana kao *plugin* za dTwin platformu. Ulazna tačka je klasa Plugin, koja implementira apstraktni interfejs `sc::IPlugin` – to je ugovor kojim dTwin platforma zna kako da pokrene plugin, dobije njegov naziv za meni, i preuzme generisane arhive (DigitalModel i VisualModel) nakon što korisnik završi rad:

```
1 class Plugin : public sc::IPlugin
2 {
3     MemoryArchiveContainer _outArchives;
4     WindowPlugin* _pWnd = nullptr;
5 public:
6     void show(gui::Window* parentWnd, MemoryArchiveContainer&
7     archives,
8     td::UINT4 wndID, const sc::IPlugin::Cleaner& cleaner,
9     const sc::IPlugin::CallBack& onComplete) override final
10    {
11        for (size_t i = 0; i < size_t(ArchType::NA); ++i)
12            _outArchives[i] = archives[i];
13
14        if (_pWnd) _pWnd->setFocus();
15        else {
16            _pWnd = new WindowPlugin(parentWnd, this, onComplete,
17            cleaner, wndID);
18            _pWnd->open();
19        }
20    }
21    // ... getMenuName(), getArchive(), getModelType(), itd.
22 };
23
24 static Plugin s_plugin;
25
26 extern "C" {
27     PLUGIN_API sc::IPlugin* getPluginInterface() { return &
28     s_plugin; }
29 }
```

Bitno je da postoji **tačno jedna** statička instanca (`s_plugin`) – ako korisnik ponovo otvori plugin dok je prozor već otvoren, `show()` samo fokusira postojeći prozor (`_pWnd->setFocus()`) umjesto da otvori duplikat.

3.1.1 Tok podataka kroz aplikaciju

Slika 3.1 prikazuje redoslijed poziva od trenutka kada korisnik odabere Matpower fajl do trenutka kada je .dmodl fajl zapisan na disk.

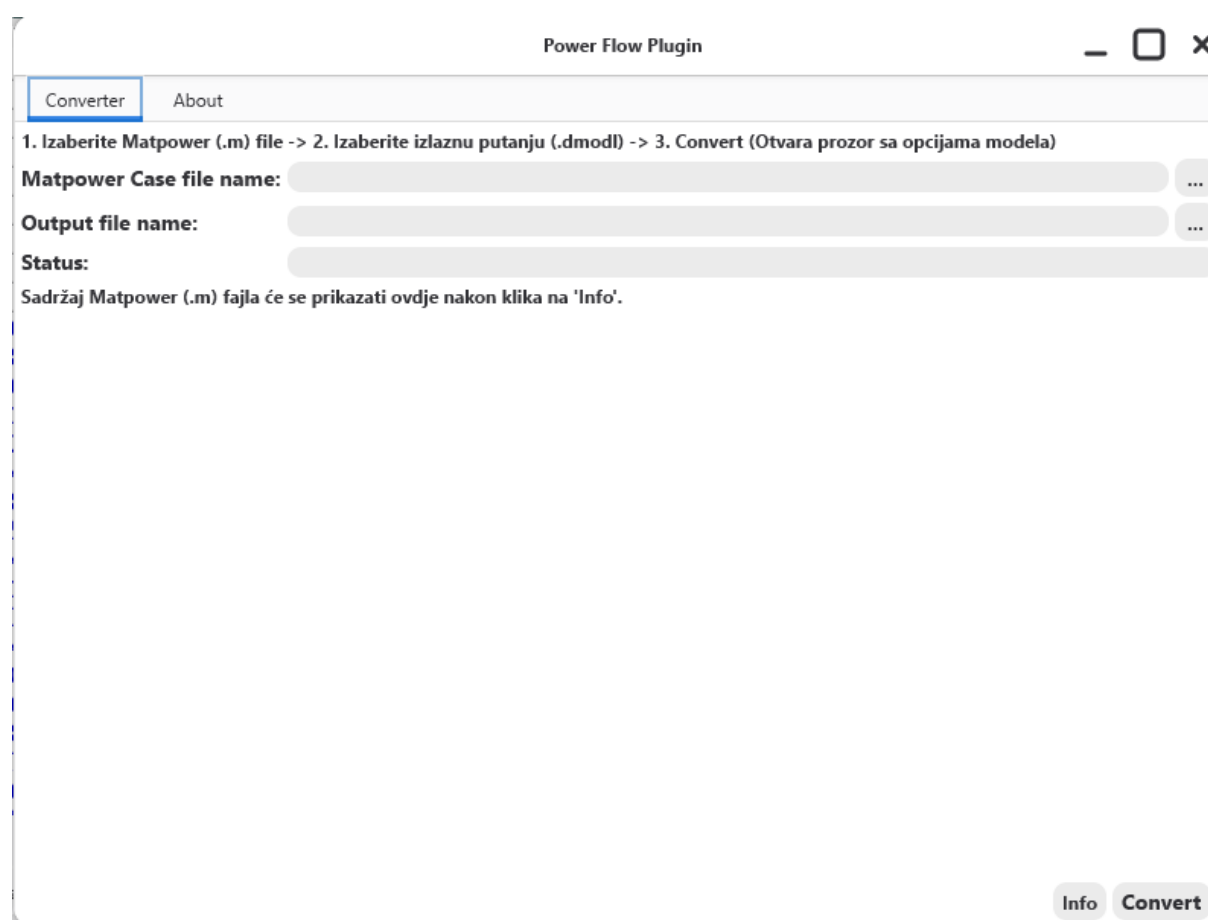


Slika 3.1: Tok podataka od odabira fajla do generisanog modela.

4 Korištenje aplikacije

4.1 Glavni prozor (Converter tab)

U početnom prozoru nalazi se nekoliko opcija koje korisnik ima.



Slika 4.1: Izgled glavnog prozora

Postoje 2 prostora za unos fajlova:

- Za unos Matpower `.m` fajla u kojem je opisan EES.
- Za unos putanje u koju će se formirati izlazni `.dmodl` fajl.

Odmah nakon odabira ulaznog fajla (prije klika na bilo koje drugo dugme), aplikacija automatski poziva `getAllBusInfo()` i ispisuje broj čvorova i generatora u statusnoj traci – korisnik tako odmah dobija povratnu informaciju da li je fajl validan, bez potrebe da prvo klikne na Info:

```
1 void previewInputFileStats(const td::String& fileName)
2 {
3     cnt::PushBackVector<int>      genBuses;
4     cnt::PushBackVector<BusInfo> allBuses;
5     if (getAllBusInfo(fileName, genBuses, allBuses, _editStatus))
6     {
7         char buf[160];
8         snprintf(buf, sizeof(buf), "INFO! Fajl OK: %d cvorova, %d
9         generatora.",
10                (int)allBuses.size(), (int)genBuses.size());
11         _editStatus = buf;
12         _editStatus.setTextColor(td::Accent::Success);
13     }
14     else
15         _editStatus.setTextColor(td::Accent::Error);
16 }
```

Ispod se nalazi *statusna traka* u kojoj se ispisuju informacije o ispravnosti unesenog fajla – u slučaju greške ispisuje se poruka o tačnom uzroku (npr. koji red fajla ne odgovara očekivanom formatu), a ako je fajl ispravan, ispisuje se broj čvorova i broj generatora.

Klikom na dugme **Info**, u polju ispod se ispisuje sirovi sadržaj .m fajla (korisno za brzu vizuelnu provjeru), dok tab **Info** (ranije *About*) sadrži statičan tekst koji objašnjava format GEN i BUS/PLOT fajlova (§4.3).

Dugme **Convert** otvara novi prozor (`WindowCreateModel`) u kojem korisnik podešava sve parametre modela prije generisanja. Ako je taj prozor već otvoren (isti `wndID`), klik na **Convert** ga samo fokusira umjesto da otvori duplikat – provjera se radi preko `pParentWnd->getAttachedWindow(wndID)`.

4.2 Prozor za kreiranje modela

Izgled prozora za podešavanje prikazan je na slici – korišten je primjer EES-a sa 9 čvorova (case9.m).

Create Model

Grafici:

☒ Ugaone brzine ☒ Mehanicke snage ☒ Rotorski uglovi ☒ Naponi

Parametri generatora (GEN id / kljuc=vrijednost / END):

Poremecaji iz fajla (BUS id / t Pd Qd / END) - opciono, ima prioritet nad rucnim unosom ispod:

Prikaz cvorova:

☒ Svi cvorovi

☒ Gen 1 (P=0,00 Q=0,00)	☒ Gen 6 (P=0,11 Q=0,07)	☒ Cvor 11 (P=0,03 Q=0,01)
☒ Gen 2 (P=0,21 Q=0,12)	☒ Cvor 7 (P=0,00 Q=0,00)	☒ Cvor 12 (P=0,06 Q=0,01)
☒ Gen 3 (P=0,94 Q=0,19)	☒ Gen 8 (P=0,00 Q=0,00)	☒ Cvor 13 (P=0,13 Q=0,05)
☒ Cvor 4 (P=0,47 Q=-0,03)	☒ Cvor 9 (P=0,29 Q=0,16)	☒ Cvor 14 (P=0,14 Q=0,05)
☒ Cvor 5 (P=0,07 Q=0,01)	☒ Cvor 10 (P=0,09 Q=0,05)	

Poremecaji tereta (manual):

Cvor	t_start	t_end	newP	newQ	Cvor	t_start	t_end	newP	newQ
☐ C1[G] P=0,00 Q=0,00	0,500000	6,000000	0,000000	0,000000	☐ C8[G] P=0,00 Q=0,00	0,500000	6,000000	0,000000	0,000000
☐ C2[G] P=0,21 Q=0,12	0,500000	6,000000	0,000000	0,000000	☐ C9 P=0,29 Q=0,16	0,500000	6,000000	0,000000	0,000000
☐ C3[G] P=0,94 Q=0,19	0,500000	6,000000	0,000000	0,000000	☐ C10 P=0,09 Q=0,05	0,500000	6,000000	0,000000	0,000000
☐ C4 P=0,47 Q=-0,03	0,500000	6,000000	0,000000	0,000000	☐ C11 P=0,03 Q=0,01	0,500000	6,000000	0,000000	0,000000
☐ C5 P=0,07 Q=0,01	0,500000	6,000000	0,000000	0,000000	☐ C12 P=0,06 Q=0,01	0,500000	6,000000	0,000000	0,000000
☐ C6[G] P=0,11 Q=0,07	0,500000	6,000000	0,000000	0,000000	☐ C13 P=0,13 Q=0,05	0,500000	6,000000	0,000000	0,000000
☐ C7 P=0,00 Q=0,00	0,500000	6,000000	0,000000	0,000000	☐ C14 P=0,14 Q=0,05	0,500000	6,000000	0,000000	0,000000

Slika 4.2: Prozor za podešavanje modela

Klasa `ViewCreateModel` je najveća i najsloženija View klasa u aplikaciji, jer njen izgled zavisi od veličine učitano sistema. Interno se sistem klasifikuje kao „mali“ (do 50 sabirnica) ili „veliki“ (preko 50 sabirnica):

```
1 _isLargeSystem(allBuses.size() > 50)
```

Za male sisteme, korisnik ima potpunu ručnu kontrolu (checkbox-ovi po sabirnici, tabela poremećaja); za velike sisteme, ručno popunjavanje stotina ili hiljada polja nije praktično, pa se poremećaji i izbor sabirnica za prikaz definišu isključivo kroz tekstualni fajl. Ipak, učitavanje fajla je od nedavno omogućeno i za male sisteme, kao brža alternativa ručnom unosu (§4.2.3).

4.2.1 Odabir veličina za iscrtavanje

Dostupne su četiri veličine za grafičko prikazivanje, svaka kao zaseban `gui::CheckBox`, grupisane u jedan horizontalni red (`gui::HorizontalLayout`) radi uštede vertikalnog prostora:

- Ugaone brzine generatora (ω)
- Mehaničke snage generatora (P_{gmi})
- Rotorski uglovi (δ)
- Naponi svih čvorova (V_t)

4.2.2 Parametri generatora (GEN fajl)

S obzirom da svaki generator može imati različite dinamičke parametre, omogućen je unos `.txt` fajla iz foldera **Generator parameters** u kojem su upisani neki ili svi parametri svakog generatora u sistemu – ova sekcija je dostupna **bez obzira** na veličinu sistema. Primjer takvog fajla (za slučaj sa 9 čvorova) dat je u nastavku:

```

1 # Generator 1 (bus 1)
2 GEN 1
3 H=23.64 D=2 Xd=0.146 Xdp=0.0608 Xq=0.0969 Xqp=0.0969
4 Tdop=8.96 Tqop=0.31 Tr=0.02 Tgov=0.3
5 Ka=200 Ta=0.02 Ke=1.0 Te=0.8 Se=0 Kf=0.063 Tf=1 R=0.05 Rs=0.0
6 END
7
8 # Generator 2 (bus 2)
9 GEN 2
10 H=6.4 D=2 Xd=0.8958 Xdp=0.1198 Xq=0.8645 Xqp=0.1969
11 Tdop=6.0 Tqop=0.535 Tr=0.02 Tgov=0.3
12 Ka=200 Ta=0.02 Ke=1.0 Te=0.8 Se=0 Kf=0.063 Tf=1 R=0.05 Rs=0.0
13 END
14
15 # Generator 3 (bus 3)
16 GEN 3
17 H=3.01 D=2 Xd=1.3125 Xdp=0.1813 Xq=1.2578 Xqp=0.25
18 Tdop=5.89 Tqop=0.6 Tr=0.02 Tgov=0.3
19 Ka=200 Ta=0.02 Ke=1.0 Te=0.8 Se=0 Kf=0.063 Tf=1 R=0.05 Rs=0.0
20 END

```

Fajl može biti i nepotpun – ako je neki parametar izostavljen, u modelu se koristi standardna (default) vrijednost. Ovo je tehnički ostvareno preko strukture `GenDynParams`, koja za svaki parametar drži par (`bool hasX`, `double X`) – zastavicu „da li je korisnik naveo ovaj parametar“ i samu vrijednost:

```

1 struct GenDynParams {
2     int    busId = 0;
3     bool   hasH = false;    double H = 0.0;
4     bool   hasD = false;    double D = 0.0;
5     bool   hasXd = false;   double Xd = 0.0;
6     // ... (isti obrazac za Xdp, Xq, Xqp, Tdop, Tqop, Tr, Tgov,

```

```
7 //      Ka, Ta, Ke, Te, Se, Kf, Tf, R, Rs)
8 };
```

Prilikom generisanja modela (funkcija `dynGenParameters()`), za svaki generator se prvo postave standardne (default) vrijednosti, a zatim se, ako fajl postoji i sadrži unos za tu sabirnicu, svaka zastavljena (`has...=true`) vrijednost **prepiše** korisnički zadanom:

```
1 double H = 6.4, D = 28; // ... standardne vrijednosti ...
2 auto it = genParamsMap.find(k);
3 if (it != genParamsMap.end())
4 {
5     const GenDynParams& p = *it->second;
6     if (p.hasH)      H = p.H;
7     if (p.hasD)      D = p.D;
8     // ... itd. za sve ostale parametre ...
9 }
```

Prilikom učitavanja, plugin (funkcija `parseGenParamsFile()`) provjerava:

- da navedena sabirnica **postoji** u sistemu;
- da na njoj **stvarno postoji generator** (nije npr. čisto opterećena sabirnica);
- da ne postoji **dupliran** GEN blok za istu sabirnicu;
- da su vremenske konstante (H , T_{dop} , T_{qop} , T_a , T_e , T_f , T_{gov}) **veće od nule**, jer se u modelu koriste kao djelilac (dijeljenje nulom bi dalo nedefinisan/beskonačan rezultat).

Ako bilo koja od ovih provjera ne prođe, učitavanje se prekida uz jasnu poruku o grešci u statusnoj traci, a već učitani parametri se ne mijenjaju.

4.2.3 Poremećaji iz fajla naspram ručnog unosa

Ovo je sekcija koja se najviše razlikuje između malih i velikih sistema.

Veliki sistemi

Za sisteme sa više od 50 sabirnica, poremećaji opterećenja i izbor sabirnica za prikaz definišu se **isključivo** kroz tekstualni fajl iz foldera *Disturbance* s obzirom da je moguće učitati fajlove sa nekoliko stotina, pa i hiljada čvorova, ne bi bilo moguće (niti praktično za korisnika) staviti hiljade polja na jedan prozor.

Mali sistemi

Za manje sisteme korisnik bira između dva potpuno ravnopravna načina:

- **Ručni unos:** za svaki čvor postoji red sa `CheckBox`-om (funkcija *Enable-a*), i, ako je označen, dva numerička polja za vremenske trenutke i dva za nove vrijednosti P_d/Q_d . U prvom trenutku teret na čvoru poprima zadanu vrijednost (simulira npr. priključenje potrošača), a u drugom se vraća na originalnu vrijednost iz Matpower fajla.

- **Fajl**: identičan format kao za velike sisteme (§4.3), ali sada dostupan i ovdje kao brža alternativa.

Ako korisnik učitava fajl za mali sistem, on ima **prednost** nad ručnim unosom – ali samo za onaj dio koji fajl stvarno definiše:

```
1 bool fileEventsActive = _useFileEvents && !_fileLoadedProfiles.  
    empty();  
2  
3 if (fileEventsActive && _fileSelectedBuses.size() != 0)  
4 {  
5     // PLOT iz fajla ima prednost nad GUI selekcijom  
6     opt.selectedBuses = ...; // iz fajla  
7 }  
8 else  
9 {  
10    // Fajl nema PLOT red - koristi GUI selekciju (checkbox/range  
    )  
11 }  
12  
13 if (fileEventsActive)  
14 {  
15     opt.loadProfiles = _fileLoadedProfiles;  
16     return opt;  
17 }  
18 // inace: koristi rucno unesene redove
```

Ovime se izbjegava neugodno iznenađenje: ako korisnik učitava fajl bez PLOT reda, izbor sabirnica za prikaz i dalje poštuje ono što je ručno označeno u GUI-u, umjesto da se tiho prebaci na „prikaži sve“.

4.3 Formati ulaznih fajlova

Aplikacija koristi dva nezavisna, jednostavna tekstualna formata – namjerno jednostavnija od punog Matpower formata, jer opisuju informacije koje Matpower format uopšte ne sadrži (dinamičke parametre generatora, vremenske poremećaje).

4.3.1 GEN fajl (dinamički parametri generatora)

Opisan detaljno u §4.2.2. Parsira ga funkcija `parseGenParamsFile()`, red po red, prepoznajući tri ključne riječi: **GEN** <busId> (otvara blok), **ključ=vrijednost** parove (mogu biti više na istom redu ili svaki u svom redu), i **END** (zatvara blok). Redovi koji počinju znakom **#** tretiraju se kao komentar.

4.3.2 PLOT/BUS fajl (poremećaji tereta)

Fajl može opciono započeti jednim redom **PLOT**, koji navodi koje će se sabirnice prikazati na svim grafikonima – ovo je gotovo neophodno za sisteme sa velikim brojem čvorova, jer bi u suprotnom prikaz bio potpuno nepregledan. Redovi koji slijede definišu poremećaje: **BUS** otvara blok za određenu sabirnicu, zatim slijedi jedan ili više redova sa vremenom i novim vrijednostima opterećenja, a **END** zatvara blok:

```

1 # Primjer fajla poremećaja tereta za case14 (n_bus=14, baseMVA
  =100)
2 # PLOT red mora biti prije svih BUS blokova.
3 PLOT 1 2 3 4 6 8 9 13 14
4
5 # Cvor 3 (orig: Pd=94.2 MW, Qd=19 MVar -> 0.942 / 0.19 pu)
6 BUS 3
7 0.5 1.3188 0.266
8 6.0 0.942 0.19
9 END
10
11 # Cvor 4 (orig: Pd=47.8 MW, Qd=-3.9 MVar -> 0.478 / -0.039 pu)
12 BUS 4
13 0.8 0.6692 -0.0546
14 3.0 0.478 -0.039
15 END
16
17 # Cvor 9 (orig: Pd=29.5 MW, Qd=16.6 MVar -> 0.295 / 0.166 pu)
18 BUS 9
19 0.5 0.413 0.2324
20 8.0 0.295 0.166
21 END

```

PLOT prihvata pojedinačne brojeve sabirnica, opsege (npr. 5-9) i kombinacije odvojene zarezom; ako se izostavi, na grafikonima se prikazuju sve sabirnice. Unutar svakog BUS bloka, svaki red predstavlja jednu vremensku tačku u obliku „vrijeme[s] Pd[pu] Qd[pu]“, a tačke se **automatski sortiraju** po vremenu bez obzira na redoslijed unosa (funkcija `parseLoadEventsFile()` to radi preko `std::sort` nad vektorom tačaka svakog bloka, prije nego što ga doda u konačnu listu). P_d i Q_d su izraženi u per-unit sistemu (MW/MVar podijeljeni sa `baseMVA`), na isti način kao i originalno opterećenje iz Matpower fajla.

Bitna razlika u odnosu na GEN fajl: BUS blokovi za **istu** sabirnicu smiju se ponoviti više puta u istom fajlu (za razliku od GEN bloka, gdje je duplikat greška) – ovo je namjerna odluka koja omogućava da se za jednu sabirnicu definiše više nezavisnih, vremenski razdvojenih scenarija (npr. poremećaj u $t = 2s$ i sasvim drugi poremećaj u $t = 30s$ na istoj sabirnici), bez potrebe da se sve tačke strpaju u jedan blok.

5 Algoritam generisanja modela

Ovo je srce aplikacije – funkcija `createModel()`, koja redom poziva niz manjih funkcija, svaka odgovorna za jedan dio `.dmodl` sadržaja. Cijeli proces se sastoji od 16 koraka; nakon svakog većeg koraka poziva se `reportProgress()`, koji (ako je zadana `onProgress` povratna funkcija) ažurira GUI traku napretka – bitno za velike sisteme, gdje generisanje modela može potrajati nekoliko sekundi.

```
1 bool createModel(const td::String& inputFileName, const td::
    String& outFileName,
2     sc::IPlugin* pIPlugin, const Options& options, gui::LineEdit&
    status,
3     const std::function<void(double)>& onProgress)
4 {
5     const int totalSteps = 16;
6     int step = 0;
7     auto reportProgress = [&]() {
8         if (onProgress) onProgress((double)(++step) / totalSteps)
9     };
10
11     mu::ScopedCLocale scopedLocale;
12     MatpowerData data;
13     if (!parseMatpower(inputFileName, data, status)) return false
14 ;
15     reportProgress();
16
17     // ... redom: modelHeader, modelVarsInit, dynGenVarsInit,
18     modelParams,
19     // dynGenParameters, submodelVarsInit, submodelParamsInit,
20     submodelNLE,
21     // submodelPostProc, modelODE, modelNLE, modelPostProc ...
22
23     // zapisivanje .dmodl fajla, zatim visualModel() i
24     zapisivanje .vmodl fajla
25 }
```

5.1 Parsiranje Matpower fajla

Funkcija `parseMatpower()` čita `.m` fajl red po red, prateći jednostavan automat sa stanjima (`ParseState::None`, `Bus`, `Gen`, `Branch`) koje se mijenja kada naide na oznake poput `mpc.bus =`, `mpc.gen =`, `mpc.branch =` ili zatvarajuću `]`; . Nekoliko posebnih slučajeva

zaslužuje pomen:

- **Detekcija kW/MW jedinica:** neki Matpower fajlovi (obično distribicioni sistemi) navode opterećenje u kilovatima, a ne megavatima, uz komentar poput % (Pd i Qd specified in kW). Parser traži podniz "kW" unutar komentara i, ako ga nađe, dijeli sva P_d/Q_d sa 1000.
- **Sanacija nedostajućeg baseKV:** ako je baseKV neke sabirnice 0 (nedostaje u fajlu), koristi se baseKV slack sabirnice kao zamjenska vrijednost.
- **Konverzija otpora/reaktanse iz Ohm-a u p.u.:** za distribucione sisteme (mali baseMVA, obično ≤ 10) gdje su otpor i reaktansa grana dati u Omima, a ne u per-unit sistemu, vrijednosti se dijele sa impedantnom bazom $Z_{base} = V_{base}^2 / S_{base}$.

5.2 Građenje Y-bus matrice

Funkcija buildYbus() gradi admitantnu (Y_{bus}) matricu iz podataka o granama, koristeći standardni π -model voda sa transformatorskim odnosom k :

```

1 static void buildYbus(const MatpowerData& data, int maxBusId,
2   dense::Matrix<double>& G, dense::Matrix<double>& B)
3 {
4     G.reserve(maxBusId + 1, maxBusId + 1, nullptr, true);
5     B.reserve(maxBusId + 1, maxBusId + 1, nullptr, true);
6     auto Gm = G.getManipulator();
7     auto Bm = B.getManipulator();
8
9     for (size_t i = 0; i < data.branches.size(); ++i)
10    {
11        // ... racunanje g_line, b_line iz r, x ...
12        Gm(f, t) -= g_line / k;      Bm(f, t) -= b_line / k;
13        Gm(t, f) -= g_line / k;      Bm(t, f) -= b_line / k;
14        Gm(f, f) += g_line / (k * k); Bm(f, f) += (b_line +
15        b_shunt / 2.0) / (k * k);
16        Gm(t, t) += g_line;          Bm(t, t) += (b_line +
17        b_shunt / 2.0);
18    }
19    // + shunt susceptanse sabirnica (Bs / baseMVA)
20 }
```

Ova funkcija je jedina implementacija Y-bus algoritma u cijelom kodu i koristi se za 4 različite funkcije: (modelParams, submodelNLE, submodelPostProc, modelNLE) što je efikasnije i jednostavnije nego da svaka ima svoj algoritam koji se ponavlja.

5.3 Generisanje .dmodl sadržaja

Preostale funkcije redom pišu tekstualni .dmodl format, opisan detaljnije u prethodnom poglavlju. Ukratko, redoslijed je:

1. **modelHeader** – solver podešavanja (`maxIter`, `dTime`, `endTime`), sada čitana iz `Options` strukture umjesto hardkodiranih vrijednosti.
2. **modelVarsInit** – početne vrijednosti mrežnih napona (e_i, f_i) za sve sabirnice, izračunate iz Matpower rješenja tokova snaga (V_m, V_a).
3. **dynGenVarsInit** – deklaracija dinamičkih stanja svakog generatora ($\delta, \omega, E'_q, E'_d, E_{fd}, R_f, V_R, P_{gmr}$).
4. **modelParams** – Y-bus elementi, injekcije snaga po sabirnici, zadani naponi za PV sabirnice.
5. **dynGenParameters** – dinamički parametri generatora (H, D, reaktanse, vremenske konstante), sa korisničkim override-om iz GEN fajla ako postoji.
6. **submodelVarsInit/ParamsInit/NLE/PostProc** – zaseban „pod-model“ koji rješava tokove snaga Newton-Raphson metodom i iz tog rješenja izvodi početne uslove svih dinamičkih stanja generatora (detaljno u §??).
7. **modelODE** – 7 diferencijalnih jednačina po generatoru (swing, fluks, AVR, governor).
8. **modelNLE** – algebarske jednačine (statorske struje, elektromagnetna snaga, naponi, mrežni bilans struje za sve sabirnice).
9. **modelPostProc** – vremenski-uslovljeni blokovi (`if t >= X: ... end`) koji implementiraju korisnički definisane poremećaje tereta.
10. **visualModel** – zaseban .vmodl fajl sa definicijama grafika (koje veličine, koje boje, koje sabirnice).

Sve funkcije dijele isti obrazac pisanja: koriste `td::MutableString` kao privremeni bafer (`mStr.appendFormat(...)`), a zatim ga upisuju u odgovarajuću memorijsku arhivu (`memDigitalOut.put(...)`) i resetuju za sljedeći red – ovo izbjegava kreiranje hiljada malih string objekata za velike sisteme.